Praktikum 3 – Program Counter, Memory Address Register dan Random Access Memory

NRP	: 1. 3223600038
	2. 3223600041
	3. 3223600048
	4. 3223600050
	5. 3223600058
Nama	: 1. Karolus Wira Wicaksana.....
	2. Akmal Rafiuddin.....
	3. Moch. Faris Afif.....
	4. Labib Al Afaf..
	5. Ihsanta Zaki Sanjaya.....
Kelas	: 2 D4 Teknik Komputer B.
Matakuliah	: Praktikum Desain dadn Organisasi Komputer
Tanggal	: Rabu, 5 Maret 2025.
Dosen	: Dr. Setiawardhana, ST. MT.

D4 TEKNIK KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2025

I. TUJUAN

a. Mahasiswa dapat menjelaskan dan membuat program Program Counter.
b. Mahasiswa dapat menjelaskan dan membuat program Memory Address Register.
c. Mahasiswa dapat menjelaskan dan membuat program Random Access Memory.

II. DASAR TEORI

A. Program Counter

Pencacah Program atau Program Counter disingkat PC adalah bagian unit kendali yang mencacah dari 0000 sampai 1111. Program disimpan pada bagian awal dari memori dengan instruksi pertama pada alamat biner 0000, intruksi kedua pada alamat 0001, instruksi ketiga pada alamat 0010, dan seterusnya. Tugasnya Program Counter adalah mengirimkan ke memori alamat dari instruksi berikutnya yang akan diambil dan dilaksanakan. Pencacah program direset ke 0000 setiap kali sebelum komputer dijalankan. Ketika komputer mulai bekerja, pencacah program mengirimkan alamat 0000 ke memori. Kemudian mencacah program meningkatkan angka cacahannya menjadi 0001. Setelah instruksi pertama diambil dan dilaksanakan, pencacah program mengirimkan alamat 0001 ke memori. Pencacah program kembali meningkatkan angka cacahannya. Sesudah instruksi kedua diambil dan dieksekusi, pencacah program mengirimkan alamat 0010 ke memori. Dengan demikian,

pencacah program dapat menentukan instuksi berikutnya yang akan diambil dan dilaksanakan. Pencacah program di sini dapat dibayangkan seperti seseorang yang memakai jarinya untuk menunjuk kepada suatu daftar instruksi ini harus dilaksanakan pertama, itu kedua, itu ketiga, dan seterusnya. Itu sebabnya mengapa pencacah program kadang-kadang disebut penunjuk (pointer): pencacah itu menunjuk kepada suatu alamat tertentu dalam memori yang menyimpan isi penting pada alamat tsb.

B. Memori Address Register

Memory address register disingkat MAR artinya register alamat memori. Register yang menyimpan sementara alamat memori untuk dikirimkan ke RAM. MAR adalah bagian dari memori SAR-1. Selama komputer bekerja, alamat dalam pencacah program ditahan (latched) pada MAR. Sejenak kemudian, MAR mengirimkan alamat 4 bit ini ke dalam RAM, dimana operasi membaca dilaksanakan.

C. Random Access Memori

Random Access Memori disingkat RAM artinya akses lokasi memori secara random. Selama komputer beroperasi, RAM menerima alamat 4-bit dari MAR dan operasi membaca dilaksanakan. Dalam proses ini, instruksi dan kata-data yang tersimpan dala RAM ditempatkan pada bus W

untuk digunakan oleh beberapa bagian lain dari komputer. Kotak RAM dalam gambar 1 merupakan sebuah RAM TTL statik 16 x 8 untuk menyimpan program dan data. Kita dapat memprogram RAM dengan register saklar alamat dan register saklar data. Melalui cara ini kita dapat memasukkan program dan data ke dalam memori sebelum komputer bekerja.

III. TUGAS PENDAHULUAN

a. Tuliskan tujuan praktikum

   Bertujuan agar dapat memahami konsep dan fungsi Program Counter, Memory Address Register, dan Random Access Memory dalam sistem komputer. Sehingga diharapkan mampu menjelaskan serta membuat program sederhana yang merepresentasikan cara kerja ketiga komponen tersebut dalam eksekusi instruksi di komputer.

b. Apa yang anda ketahui tentang Program Counter

   Program Counter adalah register dalam unit kendali yang bertugas menyimpan alamat instruksi berikutnya yang akan dieksekusi oleh prosesor. PC secara otomatis bertambah setiap kali instruksi dieksekusi, memastikan program berjalan secara berurutan. Jika terjadi percabangan dalam program, PC dapat diperbarui dengan alamat instruksi yang sesuai agar alur eksekusi dapat berubah sesuai kebutuhan.

c. Apa yang anda ketahui tentang Memori Address Register

   Memory Address Register adalah register yang menyimpan alamat memori dari data atau instruksi

yang akan diakses oleh prosesor. MAR bekerja dengan mengirimkan alamat tersebut ke RAM, sehingga data atau instruksi yang tersimpan dapat diambil dan diproses. Fungsi utama MAR adalah menjadi penghubung antara unit pemrosesan dan memori dalam proses pengambilan data atau instruksi.

d. Apa yang anda ketahui tentang Random Access Memory

Random Access Memory adalah jenis memori utama dalam komputer yang berfungsi menyimpan data dan instruksi yang sedang digunakan oleh prosesor secara sementara. RAM bersifat volatil, artinya data di dalamnya akan hilang saat daya dimatikan. Kecepatan akses RAM lebih tinggi dibanding penyimpanan sekunder, sehingga berperan penting dalam meningkatkan kinerja sistem komputer.

IV. PERCOBAAN

a. Program PC(Program Counter)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity PC is
    Port ( nCLK : in  STD_LOGIC;
           nCLR : in  STD_LOGIC;
           Cp : in  STD_LOGIC;
           Ep : in  STD_LOGIC;
           BUS_Low : out  STD_LOGIC_VECTOR (3 downto 0));
    end PC;

architecture Behavioral of PC is
    signal q: STD_LOGIC_VECTOR (3 downto 0):="0000";
begin
    process(nCLK, nCLR, Cp,Ep)
    begin
        if rising_edge(nCLK) then
            if Cp = '1' then q <= q+1;          end if;
            if nCLR = '1' then q <= "0000";     end if;
        end if;
    end process;
    BUS_Low <= q when Ep='1' else "ZZZZ";
end Behavioral;
```

b. Program Memori Address Register(MAR)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MAR is
    Port ( CLK : in  STD_LOGIC;
           nLm : in  STD_LOGIC;
           inMAR : in  STD_LOGIC_VECTOR (3 downto 0);
           outMAR : out  STD_LOGIC_VECTOR (3 downto 0));
end MAR;

architecture Behavioral of MAR is
      signal mar : STD_LOGIC_VECTOR (3 downto 0);
begin
      process(CLK, nLm)
      begin
            if rising_edge(CLK) then
                  if nLm = '0' then outMAR <= inMAR; end if;
            end if;
      end process;
end Behavioral;
```

c. Program Random Access Memory (RAM)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RAM is
    Port ( nCe : in  STD_LOGIC;
           inRAM : in  STD_LOGIC_VECTOR (3 downto 0);
           outRAM : out  STD_LOGIC_VECTOR (7 downto 0));
end RAM;

architecture Behavioral of RAM is
      signal ROM : STD_LOGIC_VECTOR (7 downto 0);
begin
      process(inRAM)
      begin
            case inRAM is
                  when "0000" => ROM <= "00001001";-- LDA   9h
                  when "0001" => ROM <= "00011010";-- ADD   10h
                  when "0010" => ROM <= "00101011";-- SUBB  11h
                  when "0011" => ROM <= "11100000";-- OUT
                  when "0100" => ROM <= "11110000";-- Halt
                  when "0101" => ROM <= "00000000";
                  when "0110" => ROM <= "00000001";
                  when "0111" => ROM <= "00000010";
                  when "1000" => ROM <= "00000011";
                  when "1001" => ROM <= "00000111";   -- 7
                  when "1010" => ROM <= "00000010";   -- 2
                  when "1011" => ROM <= "00000101";   -- 5
                  when "1100" => ROM <= "00000010";
                  when "1101" => ROM <= "00000000";
                  when "1110" => ROM <= "00000000";
                  when "1111" => ROM <= "11110000";
                  when others => ROM <= "00000000";
            end case;
      end process;
      outRAM <= ROM when nCe='0' else "ZZZZZZZZ";
end Behavioral;
```

V. PROGRAM

a. Program PC (Program Counter)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity PC is
	Port ( nCLK : in STD_LOGIC;
		nCLR : in STD_LOGIC;
		Cp : in STD_LOGIC;
		Ep : in STD_LOGIC;
		BUS_Low : out STD_LOGIC_VECTOR (3
downto 0));
	end PC;
architecture Behavioral of PC is
	signal q: STD_LOGIC_VECTOR (3 downto 0):="0000";
begin
	process(nCLK, nCLR, Cp,Ep)
	begin
		if rising_edge(nCLK) then
			if Cp = '1' then q <= q+1;
			end if;
			if nCLR = '1' then q <= "0000";
			end if;
		end if;
	end process;
	BUS_Low <= q when Ep='1' else "ZZZZ";
end Behavioral;
```

Program Testbench (PC)

```
ENTITY PCTB IS
END PCTB;
ARCHITECTURE behavior OF PCTB IS

-- Component Declaration for the Unit Under Test (UUT)
COMPONENT PC
PORT(
  nCLK     : IN  std_logic;
  nCLR     : IN  std_logic;
  Cp       : IN  std_logic;
  Ep       : IN  std_logic;
  BUS_Low  : OUT std_logic_vector(3 downto 0)
);
END COMPONENT;

-- Inputs
signal nCLK     : std_logic := '0';
signal nCLR     : std_logic := '0';
signal Cp       : std_logic := '0';
signal Ep       : std_logic := '0';

-- Outputs
signal BUS_Low  : std_logic_vector(3 downto 0);

-- Clock period definitions
constant nCLK_period : time := 10 ns;

BEGIN

-- Instantiate the Unit Under Test (UUT)
uut: PC
PORT MAP (
  nCLK     => nCLK,
  nCLR     => nCLR,
  Cp       => Cp,
  Ep       => Ep,
```

```
  BUS_Low  => BUS_Low
);

-- Clock process
nCLK_process : process
begin
  nCLK <= '0';
  wait for nCLK_period / 2;
  nCLK <= '1';
  wait for nCLK_period / 2;
end process;

-- Stimulus process
stim_proc: process
begin
  wait for 100 ns;
  Ep <= not Ep;
  Cp <= not Cp;
  wait for nCLK_period * 10;
  -- insert stimulus here
  wait;
end process;

END behavior;
```

Program ucf (PC)

```
NET "nCLK" LOC = "P55";
NET "nCLR" LOC = "P82";
NET "Cp" LOC = "P81";
NET "Ep" LOC = "P80";
NET "BUS_Low[0]" LOC = "P17";
NET "BUS_Low[1]" LOC = "P16";
NET "BUS_Low[2]" LOC = "P15";
NET "BUS_Low[3]" LOC = "P14";
```

b. Memory Address Register (MAR)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MAR is
	Port ( CLK : in STD_LOGIC;
		nLm : in STD_LOGIC;
		inMAR : in STD_LOGIC_VECTOR (3 downto 0);
		outMAR : out STD_LOGIC_VECTOR (3 downto
0));
end MAR;

architecture Behavioral of MAR is
	signal mar : STD_LOGIC_VECTOR (3 downto 0);
begin
	process(CLK, nLm)
	begin
		if rising_edge(CLK) then
			if nLm = '0' then outMAR <= inMAR;
end if;
		end if;
	end process;
end Behavioral;
```

Program Testbench (MAR)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
-- Uncomment the following library declaration if using
arithmetic functions with Signed or Unsigned values
-- USE ieee.numeric_std.ALL;

ENTITY MAR_TB IS
END MAR_TB;

ARCHITECTURE behavior OF MAR_TB IS

  -- Component Declaration for the Unit Under Test (UUT)
  COMPONENT MAR
```

```
    PORT(
      CLK     : IN  std_logic;
      nLm     : IN  std_logic;
      inMAR   : IN  std_logic_vector(3 downto 0);
      outMAR  : OUT std_logic_vector(3 downto 0)
    );
  END COMPONENT;

  -- Inputs
  signal CLK     : std_logic := '0';
  signal nLm     : std_logic := '0';
  signal inMAR   : std_logic_vector(3 downto 0) := (others =>
'0');

  -- Outputs
  signal outMAR  : std_logic_vector(3 downto 0);

  -- Clock period definitions
  constant CLK_period : time := 10 ns;

BEGIN

  -- Instantiate the Unit Under Test (UUT)
  uut: MAR
    PORT MAP (
      CLK     => CLK,
      nLm     => nLm,
      inMAR   => inMAR,
      outMAR  => outMAR
    );

  -- Clock process definitions
  CLK_process : process
  begin
    CLK <= '0';
    wait for CLK_period / 2;
    CLK <= '1';
```

```
    wait for CLK_period / 2;
  end process;

  -- Stimulus process
  stim_proc0: process
  begin
    wait for 50 ns;
    nLm <= not nLm;
  end process;

  stim_proc: process
  begin
    wait for 50 ns;
    inMAR(0) <= not inMAR(0);
  end process;

  stim_proc1: process
  begin
    wait for 100 ns;
    inMAR(1) <= not inMAR(1);
  end process;

  stim_proc2: process
  begin
    wait for 150 ns;
    inMAR(2) <= not inMAR(2);
  end process;

  stim_proc3: process
  begin
    wait for 200 ns;
    inMAR(3) <= not inMAR(3);
  end process;

END behavior;
```

Program ucf (MAR)

```
NET "CLK" LOC = "P55";
NET "nLm" LOC = "P82";
NET "inMAR[0]" LOC = "P81";
NET "inMAR[1]" LOC = "P80";
NET "inMAR[2]" LOC = "P79";
NET "inMAR[3]" LOC = "P78";
NET "outMAR[0]" LOC = "P17";
NET "outMAR[1]" LOC = "P16";
NET "outMAR[2]" LOC = "P15";
NET "outMAR[3]" LOC = "P14";
```

c. Random Access Memory (RAM)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RAM is
   Port (
      nCe    : in  STD_LOGIC;
      inRAM  : in  STD_LOGIC_VECTOR (3 downto 0);
      outRAM : out STD_LOGIC_VECTOR (7 downto 0)
   );
end RAM;

architecture Behavioral of RAM is
   signal ROM : STD_LOGIC_VECTOR (7 downto 0);
begin
   process(inRAM)
   begin
      case inRAM is
         when "0000" => ROM <= "00001001"; -- LDA
         when "0001" => ROM <= "00011010"; -- ADD
         when "0010" => ROM <= "00101011"; -- SUBB
         when "0011" => ROM <= "11100000"; -- OUT
         when "0100" => ROM <= "11110000"; -- Halt
         when "0101" => ROM <= "00000000";
         when "0110" => ROM <= "00000001";
         when "0111" => ROM <= "00000010";
```

```
            when "1000" => ROM <= "00000011";
            when "1001" => ROM <= "00000111"; -- 7
            when "1010" => ROM <= "00000010"; -- 2
            when "1011" => ROM <= "00000101"; -- 5
            when "1100" => ROM <= "00000010";
            when "1101" => ROM <= "00000000";
            when "1110" => ROM <= "00000000";
            when "1111" => ROM <= "11110000";
            when others => ROM <= "00000000";
        end case;
    end process;

    outRAM <= ROM when nCe = '0' else "ZZZZZZZZ";
end Behavioral;
```

Program Testbench (RAM)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
-- USE ieee.numeric_std.ALL;

ENTITY RAM_TB IS
END RAM_TB;

ARCHITECTURE behavior OF RAM_TB IS

  -- Component Declaration for the Unit Under Test (UUT)
  COMPONENT RAM
    PORT(
      nCe    : IN  std_logic;
      inRAM  : IN  std_logic_vector(3 downto 0);
      outRAM : OUT std_logic_vector(7 downto 0)
    );
  END COMPONENT;

  -- Inputs
```

```
  signal nCe   : std_logic := '0';
  signal inRAM : std_logic_vector(3 downto 0) := (others =>
'0');

  -- Outputs
  signal outRAM : std_logic_vector(7 downto 0);

BEGIN

  -- Instantiate the Unit Under Test (UUT)
  uut: RAM
    PORT MAP (
      nCe    => nCe,
      inRAM  => inRAM,
      outRAM => outRAM
    );

  -- Stimulus process
  stim_proc: process
  begin
    -- hold reset state for 50 ns
    wait for 50 ns;
    inRAM(0) <= not inRAM(0);
  end process;

  stim_proc1: process
  begin
    -- hold reset state for 100 ns
    wait for 100 ns;
    inRAM(1) <= not inRAM(1);
  end process;

  stim_proc2: process
  begin
    -- hold reset state for 200 ns
    wait for 200 ns;
```

```
    inRAM(2) <= not inRAM(2);
  end process;

  stim_proc3: process
  begin
    -- hold reset state for 400 ns
    wait for 400 ns;
    inRAM(3) <= not inRAM(3);
  end process;

  stim_proc4: process
  begin
    -- hold reset state for 50 ns
    wait for 50 ns;
    nCe <= not nCe;
  end process;

END behavior;
```

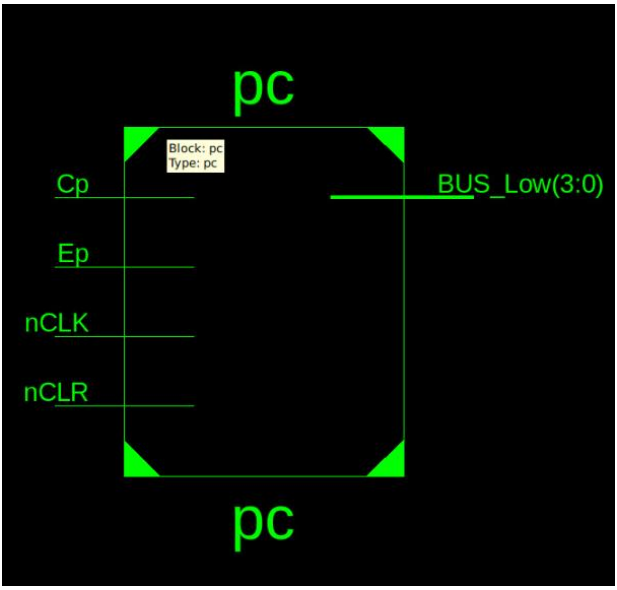
Program ucf (RAM)

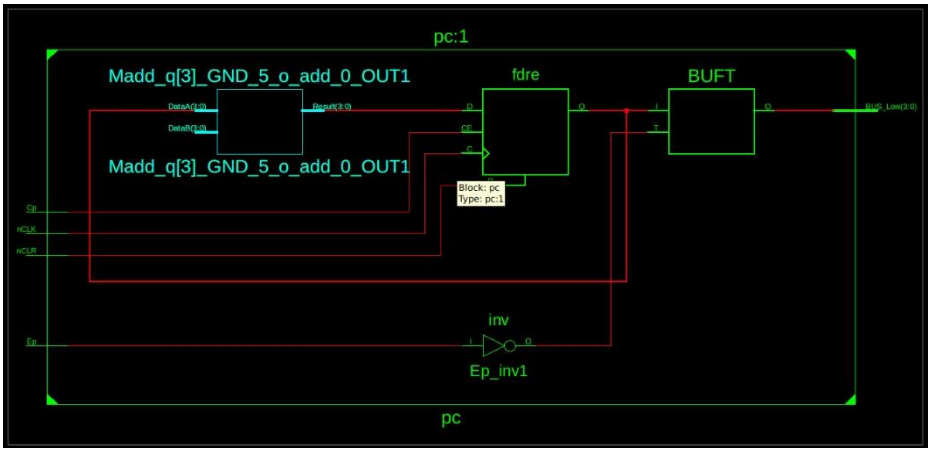
```
NET "nCe" LOC = "P82";
NET "inRAM[0]" LOC = "P81";
NET "inRAM[1]" LOC = "P80";
NET "inRAM[2]" LOC = "P79";
NET "inRAM[3]" LOC = "P78";
NET "outRAM[0]" LOC = "P17";
NET "outRAM[1]" LOC = "P16";
NET "outRAM[2]" LOC = "P15";
NET "outRAM[3]" LOC = "P14";
NET "outRAM[4]" LOC = "P13";
NET "outRAM[5]" LOC = "P12";
NET "outRAM[6]" LOC = "P11";
NET "outRAM[7]" LOC = "P10";
```

Hasil

a. Program Counter (PC)

RTL Schematics

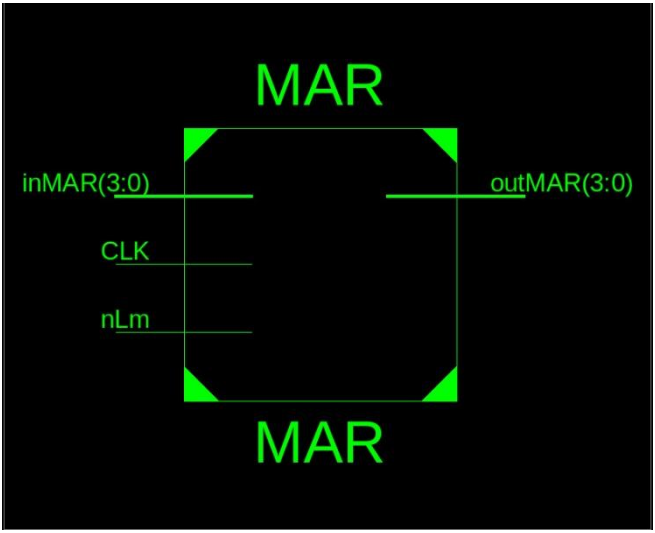


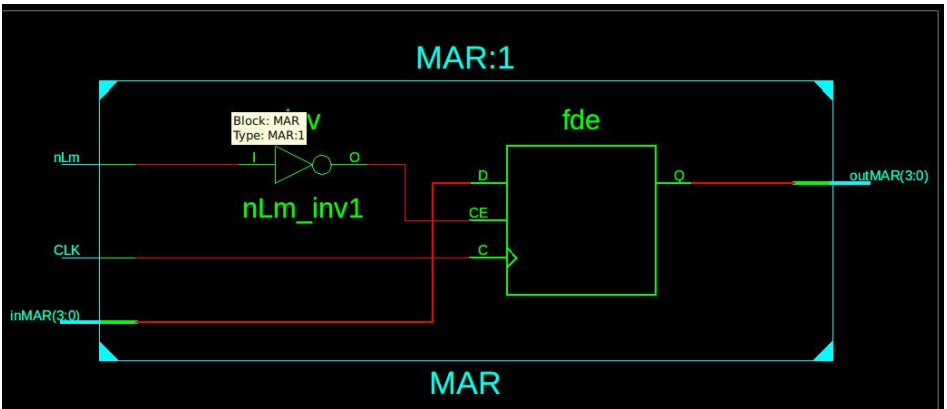


Timing Diagram

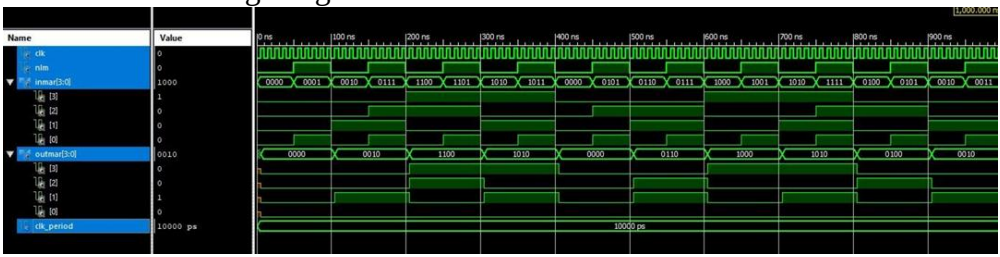
b. MAR

RTL Schematics



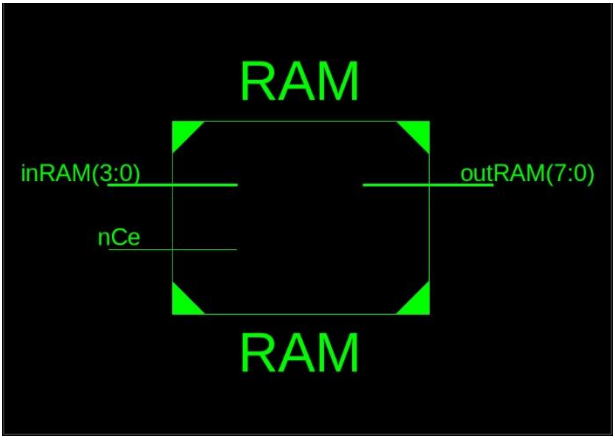


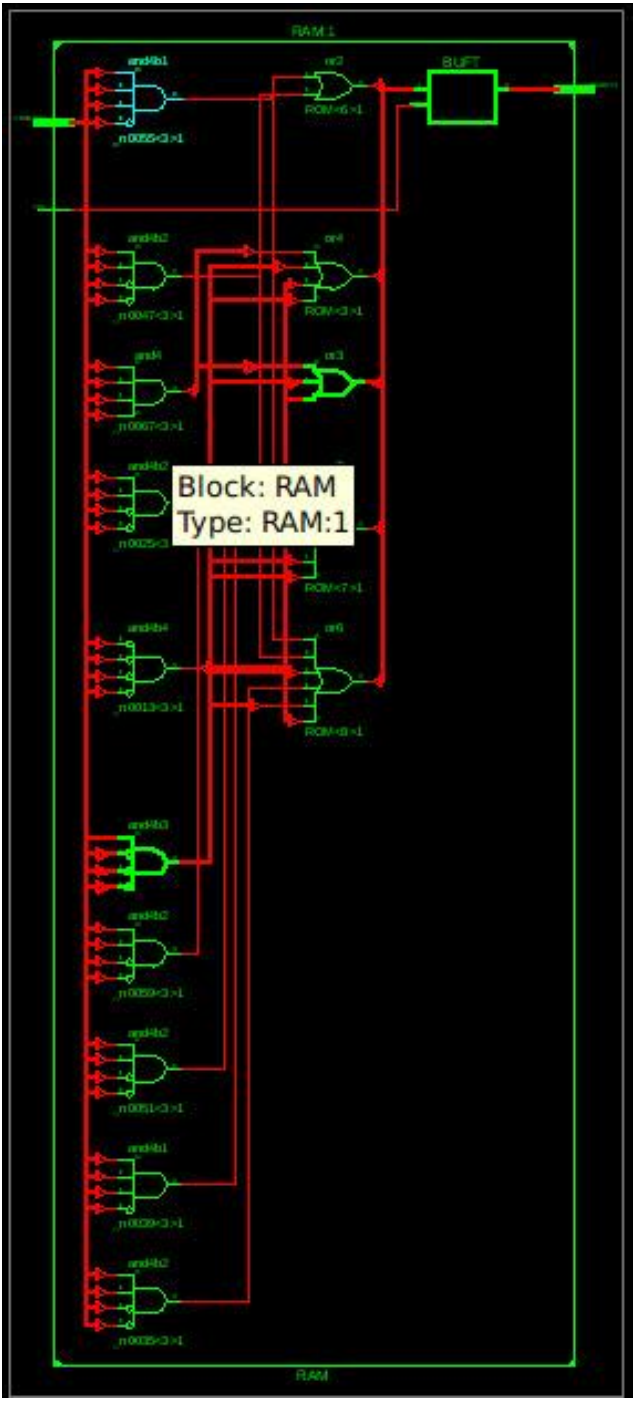
Timing Diagram



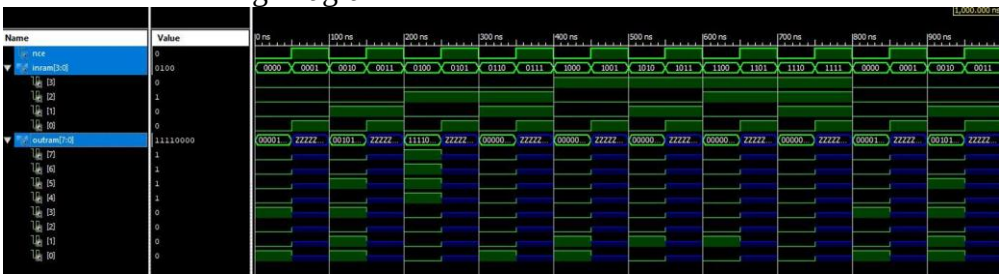
c. RAM

RTL Schematics




RAM:1
BUFT
Block: RAM
Type: RAM:1
RAM

Timing Diagram



VI. ANALISA

a. PC

Pada percobaan Program Counter, dibuat untuk menjalankan fungsi pencacah alamat instruksi. Program Counter bertanggung jawab untuk menyimpan alamat instruksi yang sedang dieksekusi dan meningkatkannya secara otomatis setiap siklus clock, kecuali ada kondisi tertentu seperti reset. Dari kode yang diimplementasikan, terdapat beberapa sinyal kontrol utama, yaitu nCLK (clock), nCLR (reset aktif tinggi), Cp (count enable), dan Ep (enable output ke bus).

Saat terjadi rising edge pada sinyal clock, jika sinyal Cp aktif (bernilai 1), maka nilai register q akan bertambah 1, yang berarti Program Counter maju ke alamat berikutnya. Namun, jika sinyal nCLR aktif (bernilai 1), maka register q akan di-reset ke "0000", yang berarti Program Counter kembali ke awal. Selain itu, output dari PC dikontrol oleh sinyal Ep. Jika Ep aktif, maka nilai dari register q akan diteruskan ke BUS_Low, memungkinkan perangkat lain membaca alamat instruksi saat ini. Jika Ep tidak aktif, maka bus berada dalam kondisi high-impedance ("ZZZZ"), yang berfungsi untuk menghindari konflik data pada sistem bus bersama.

Dari hasil gambar rangkaian, terlihat bahwa struktur utama Program Counter terdiri dari register flip-flop (fdre), buffer tri-state (BUFT), dan logic inverter (inv). Register flip-flop berfungsi sebagai penyimpan nilai counter, sedangkan buffer tri-state mengontrol kapan data bisa dikirim ke bus. Logic inverter digunakan untuk membalik sinyal kontrol tertentu agar berfungsi sesuai dengan kebutuhan rangkaian. Sehingga PC dapat bekerja dengan benar dan dapat menghitung alamat instruksi, mereset saat diperlukan, serta mengontrol output ke bus dengan mekanisme tri-state.

b. MAR

Pada percobaan Memory Address Register, rangkaian dirancang untuk menyimpan dan meneruskan alamat memori yang akan diakses. MAR berfungsi sebagai register yang menyimpan sementara alamat dari instruksi atau data yang sedang diambil dari atau ditulis ke dalam memori utama. Pada desain ini, terdapat tiga sinyal utama, yaitu CLK (clock), nLm (load enable aktif rendah), dan inMAR (data input), serta output outMAR, yang menyimpan nilai alamat saat ini.

Saat terjadi rising edge pada sinyal clock, jika nLm dalam kondisi aktif (bernilai 0), maka nilai inMAR akan disimpan ke dalam register MAR dan diteruskan ke outMAR. Sebaliknya, jika nLm bernilai 1, nilai di MAR tidak berubah, sehingga mempertahankan alamat sebelumnya. Mekanisme ini memastikan bahwa MAR hanya diperbarui saat diperlukan, sehingga menghindari perubahan alamat yang tidak disengaja selama operasi pemrosesan data.

Dari hasil gambar rangkaian, terlihat bahwa MAR terdiri dari flip-flop register (fde) dan gerbang inverter (inv).

Inverter digunakan untuk membalik sinyal nLm, sehingga register hanya akan menerima nilai baru ketika nLm aktif (0). Register fde berfungsi menyimpan alamat dan meneruskannya ke outMAR. Sehingga MAR dapat bekerja dengan benar dan mampu menyimpan alamat memori secara sementara serta hanya memperbaruinya ketika sinyal kontrol nLm aktif.

c. RAM

Pada percobaan Random Access Memory, rangkaian dirancang untuk menyimpan dan mengakses data berdasarkan alamat yang diberikan. RAM memiliki input berupa alamat (inRAM) dengan lebar 4 bit dan menghasilkan output (outRAM) dengan lebar 8 bit. Selain itu, terdapat sinyal kontrol nCe (chip enable) yang menentukan apakah data akan diakses atau tidak. Ketika nCe bernilai 0 (aktif), RAM akan mengeluarkan data yang sesuai dengan alamat yang diberikan melalui inRAM. Data yang dikeluarkan bergantung pada pemetaan nilai dalam proses VHDL yang menggunakan pernyataan case untuk menentukan nilai output berdasarkan alamat yang diberikan. Sebagai contoh, alamat "0000" akan mengeluarkan data "00001001" (instruksi LDA), sedangkan alamat "0001" akan mengeluarkan data "00011010" (instruksi ADD), dan seterusnya. Namun, ketika nCe bernilai 1, output outRAM berada dalam kondisi high-impedance ("ZZZZZZZZ"), yang berarti tidak ada data yang dikirimkan ke bus.

Berdasarkan hasil rangkaian yang ditunjukkan pada gambar, terlihat bahwa RAM diimplementasikan menggunakan beberapa gerbang logika dan multiplexer yang mengatur jalur data dari alamat yang diberikan ke output. Flip-flop digunakan untuk menyimpan nilai sementara sebelum dikeluarkan sebagai output. Setiap

kombinasi alamat akan mengaktifkan jalur tertentu di dalam rangkaian, memungkinkan data yang sesuai untuk dikirimkan ke outRAM. Sehingga RAM dapat menyimpan dan mengeluarkan data sesuai dengan alamat yang diberikan. Penggunaan sinyal nCe juga berfungsi sebagai mekanisme kontrol untuk mengaktifkan atau menonaktifkan RAM dalam sistem serta data dapat diakses secara langsung berdasarkan alamat tanpa harus melalui urutan tertentu.

VII. KESIMPULAN

Penerapan PC, MAR, dan RAM dalam praktikum ini menunjukkan bagaimana komponen-komponen dasar dalam arsitektur komputer bekerja secara terintegrasi untuk menangani instruksi dan data. PC berfungsi sebagai pencacah alamat instruksi yang memastikan eksekusi program berjalan secara berurutan, MAR menyimpan alamat memori sementara untuk mengakses data atau instruksi, sedangkan RAM menyediakan penyimpanan data yang dapat diakses langsung berdasarkan alamat yang diberikan. Hasil analisis menunjukkan bahwa masing-masing komponen bekerja sesuai fungsinya dengan memanfaatkan sinyal kontrol untuk memastikan operasi yang tepat dan efisien dalam sistem.